Dossier de conception

BookHub

Plateforme de gestion de la bibliothèque associative « Quartier Solidaire »

Projet Fullstack Java Angular –

Sprint 0 – Livrables de conception (J2)

Version 2 – octobre 2026

Sommaire

1. Introduction....3
1.1 Objet du document....3
1.2 Contexte....3
1.3 Objectifs et indicateurs de succès....3
1.4 Périmètre....3
1.5 Choix techniques....4
2. Analyse des besoins....4
2.1 Rôles et acteurs....4
2.2 Matrice des droits....5
2.3 Règles de gestion....5
3. Modélisation UML....7
3.1 Diagramme de cas d'utilisation....7
3.2 Diagramme de séquence : connexion....8
3.3 Diagramme d'architecture....9
3.4 Diagramme de classes....10
4. Modèle de données....11
4.1 Dictionnaire de données....11
4.2 Modèle conceptuel de données (MCD)....14
4.3 Modèle logique de données (MLD)....14
4.4 Modèle physique de données (MPD)....15
4.5 Triggers et fonctions PostgreSQL....16

1. Introduction

1.1 Objet du document

Ce dossier rassemble les livrables de conception back-end et base de données attendus en fin de Sprint 0 (J2) : diagrammes UML (cas d'utilisation, séquence, classes), schéma d'architecture logicielle et modèle de données (dictionnaire de données, MCD, MLD, MPD).

Les user stories avec critères d'acceptation et les maquettes (wireframes) font l'objet d'un livrable séparé, réalisé par un autre membre de l'équipe.

Ce dossier sert de référence pour les développements des Sprints 1 et 2 et pour la validation finale.

1.2 Contexte

L'association « Quartier Solidaire » (loi 1901, 3 salariés, 15 bénévoles) gère depuis 15 ans une bibliothèque communautaire de 2 000 ouvrages pour environ 300 adhérents actifs (450 au total). La bibliothèque est tenue par un bibliothécaire à temps partiel et 4 bénévoles, du mardi au samedi de 14 h à 19 h. La gestion repose aujourd'hui sur des fiches cartonnées et des registres manuscrits.

Le tableau suivant résume les problèmes actuels et leur priorité.

Problématique	Impact	Priorité
Gestion manuelle des emprunts	Erreurs, perte de temps (30 min/jour)	Haute
Pas de catalogue en ligne	Impossibilité de consulter à distance	Haute
Pas de système de réservation	Déplacements inutiles des adhérents	Moyenne
Difficultés à gérer les retards	Relances manuelles chronophages	Haute
Pas de statistiques	Impossibilité de piloter l'activité	Basse

1.3 Objectifs et indicateurs de succès

L'objectif général est de créer une plateforme web complète, accessible et éco-conçue pour la gestion de la bibliothèque associative : catalogue en ligne 24/7, emprunts et réservations automatisés, notation et commentaires, tableaux de bord de pilotage et mise en conformité RGPD.

Indicateur	Cible
Adoption de la plateforme par les adhérents	80 % après 6 mois
Temps de gestion quotidien	− 50 %
Satisfaction des utilisateurs	Note ≥ 4 / 5
Temps de chargement d'une page	< 2 s
Temps de réponse API / recherche	< 500 ms / < 1 s
Couverture de tests unitaires	≥ 20 % backend, ≥ 20 % frontend

1.4 Périmètre

Le MVP couvre les six modules fonctionnels définis par le cahier des charges (§ 4.1).

Module	Contenu
1. Authentification	Inscription, connexion, gestion du profil, rôles USER, LIBRARIAN, ADMIN

Module	Contenu
2. Catalogue	Liste et détail des livres, recherche et filtres, CRUD bibliothécaire
3. Emprunt	Emprunter un livre, mes emprunts, enregistrer un retour
4. Réservation	Réserver un livre, file d'attente
5. Notation	Noter (1 à 5 étoiles), commenter, modération
6. Dashboards	Tableaux de bord lecteur, bibliothécaire et administrateur

1.5 Choix techniques

Les technologies retenues respectent le cahier des charges (§ 6.1), à une exception près : la base de données est hébergée sur Neon (PostgreSQL) au lieu de SQL Server. Ce choix a été validé.

Couche	Technologie	Remarque
Frontend	Angular 17+, TypeScript 5+, CSS, HttpClient	Conforme au CDC
Backend	Java 21, Spring Boot 3, Spring Security + JWT, Spring Data JPA	Conforme au CDC (Java 17+)
Base de données	Neon (PostgreSQL serverless)	Remplace SQL Server 2019+ ; choix validé
Documentation API	Swagger (OpenAPI)	Conforme au CDC

2. Analyse des besoins

Cette partie décrit les acteurs de l'application, leurs droits et les règles de gestion issues du cahier des charges.

2.1 Rôles et acteurs

Acteur	Persona	Besoins principaux	Fréquence
Visiteur	Habitant du quartier non inscrit	Découvrir le catalogue, créer un compte	Ponctuelle
Lecteur (USER)	Marie, 32 ans, professeure, passionnée de lecture	Consulter le catalogue depuis chez elle, emprunter facilement, être prévenue des retards	Plusieurs fois par mois
Bibliothécaire (LIBRARIAN)	Ahmed, 45 ans, responsable de la bibliothèque	Gérer le catalogue, valider emprunts et retours, suivre les retards	Quotidienne
Administrateur (ADMIN)	Sophie, 50 ans, présidente de l'association	Vue d'ensemble, gestion des utilisateurs, statistiques	Hebdomadaire
Système (acteur secondaire)	Tâche planifiée quotidienne	Passer les emprunts échus au statut « en retard »	Quotidienne (00 h 05)

Les rôles sont hiérarchiques : un bibliothécaire dispose de tous les droits d'un lecteur, un administrateur de tous les droits d'un bibliothécaire.

2.2 Matrice des droits

Fonctionnalité	Visiteur	USER	LIBRARIAN	ADMIN
Consulter et rechercher dans le catalogue	✔	✔	✔	✔
S'inscrire, se connecter	✔	–	–	–
Gérer son profil, supprimer son compte		✔	✔	✔
Emprunter, réserver, annuler une réservation		✔	✔	✔
Noter et commenter un livre emprunté		✔	✔	✔
Tableau de bord lecteur		✔	✔	✔
Ajouter et modifier un livre			✔	✔
Consulter tous les emprunts, enregistrer un retour			✔	✔
Modérer (supprimer) un commentaire			✔	✔
Tableau de bord bibliothèque			✔	✔
Supprimer un livre				✔
Gérer les utilisateurs et leurs rôles				✔

2.3 Règles de gestion

Les règles ci-dessous proviennent du cahier des charges ; la colonne de droite indique leur origine. Une seule règle (RG-RATE-05) est un choix de conception.

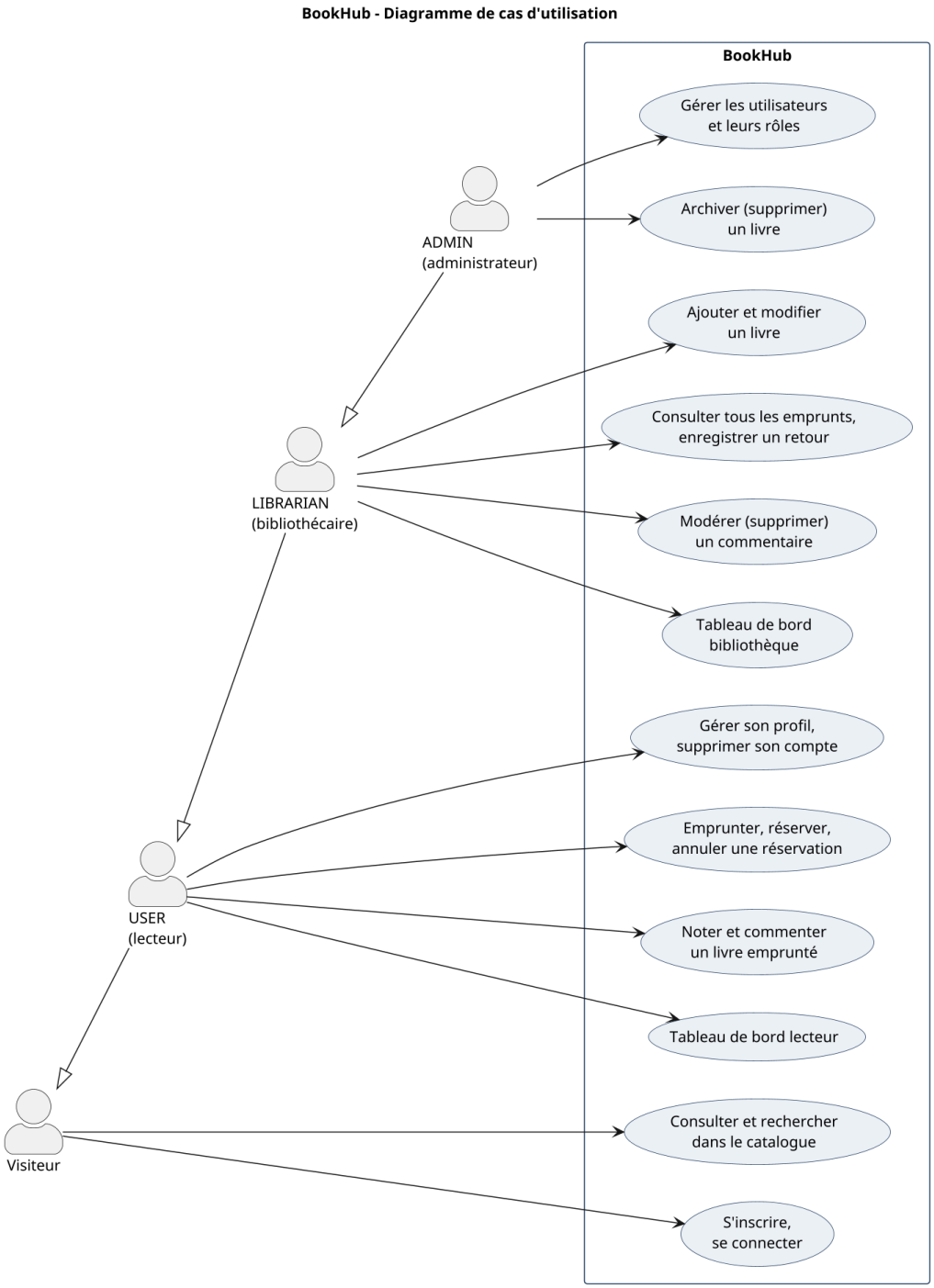
Code	Règle	Référence CDC
RG-AUTH-01	L'adresse e-mail est unique et respecte le format standard.	§ 5.1, US-AUTH-01
RG-AUTH-02	Mot de passe : 12 caractères minimum, dont 1 majuscule, 1 minuscule, 1 chiffre et 1 caractère spécial ; haché en BCrypt (coût 12).	§ 7.2, US-AUTH-01
RG-AUTH-03	Tout nouveau compte reçoit le rôle USER.	US-AUTH-01
RG-AUTH-04	Le jeton JWT (HS256, secret ≥ 256 bits) expire après 24 h ; il contient l'e-mail et le rôle.	§ 7.1, US-AUTH-02
RG-AUTH-05	L'inscription exige le consentement au traitement des données : case à cocher obligatoire, non pré-cochée, vérifiée côté client et côté serveur. Sans elle, le compte n'est pas créé.	§ 7.4 (RGPD)
RG-AUTH-06	Le lecteur peut modifier son prénom, son nom et son téléphone, changer son mot de passe et supprimer son compte (droit à l'oubli).	§ 7.4, US-AUTH-03
RG-BOOK-01	Le catalogue affiche 20 livres par page.	US-BOOK-01
RG-BOOK-02	La recherche porte sur le titre, l'auteur et l'ISBN, sans tenir compte de la casse ; filtres par catégorie et disponibilité.	US-BOOK-03
RG-BOOK-03	Seul l'administrateur peut supprimer un livre, et uniquement s'il n'a aucun emprunt.	§ 6.2, US-BOOK-04
RG-LOAN-01	Un lecteur a au maximum 3 emprunts simultanés.	§ 5.3
RG-LOAN-02	La durée d'un emprunt est de 14 jours.	§ 5.3
RG-LOAN-03	Un lecteur en retard ne peut plus emprunter.	§ 5.3
RG-LOAN-04	Un même livre peut être ré-emprunté.	§ 5.3
RG-LOAN-05	On ne peut emprunter que si un exemplaire est disponible ; l'emprunt décrémente le stock, le retour l'incrémente.	US-LOAN-01, US-LOAN-03
RG-LOAN-06	Le retard est calculé automatiquement au retour ; l'emprunt passe au statut RETURNED.	US-LOAN-03
RG-RESA-01	Un lecteur a au maximum 5 réservations.	US-RESA-01

Code	Règle	Référence CDC
RG-RESA-02	On ne peut réserver qu'un livre indisponible.	US-RESA-01
RG-RESA-03	Le rang dans la file d'attente est affiché ; une réservation est en Attente, Dispo ou Annulée, et peut être annulée par le lecteur.	US-RESA-01, US-RESA-02
RG-RATE-01	Seul un lecteur ayant emprunté le livre peut le noter, de 1 à 5 étoiles.	US-RATE-01
RG-RATE-02	La note moyenne du livre est calculée et affichée ; la note et le commentaire sont modifiables par leur auteur.	US-RATE-01, US-RATE-02
RG-RATE-03	Le commentaire fait 1000 caractères au maximum et s'affiche sous la fiche du livre.	US-RATE-02
RG-RATE-04	Le bibliothécaire peut modérer (supprimer) un avis.	§ 6.2, US-RATE-02
RG-RATE-05	Un lecteur ne peut laisser qu'un seul avis par livre.	Choix de conception

3. Modélisation UML

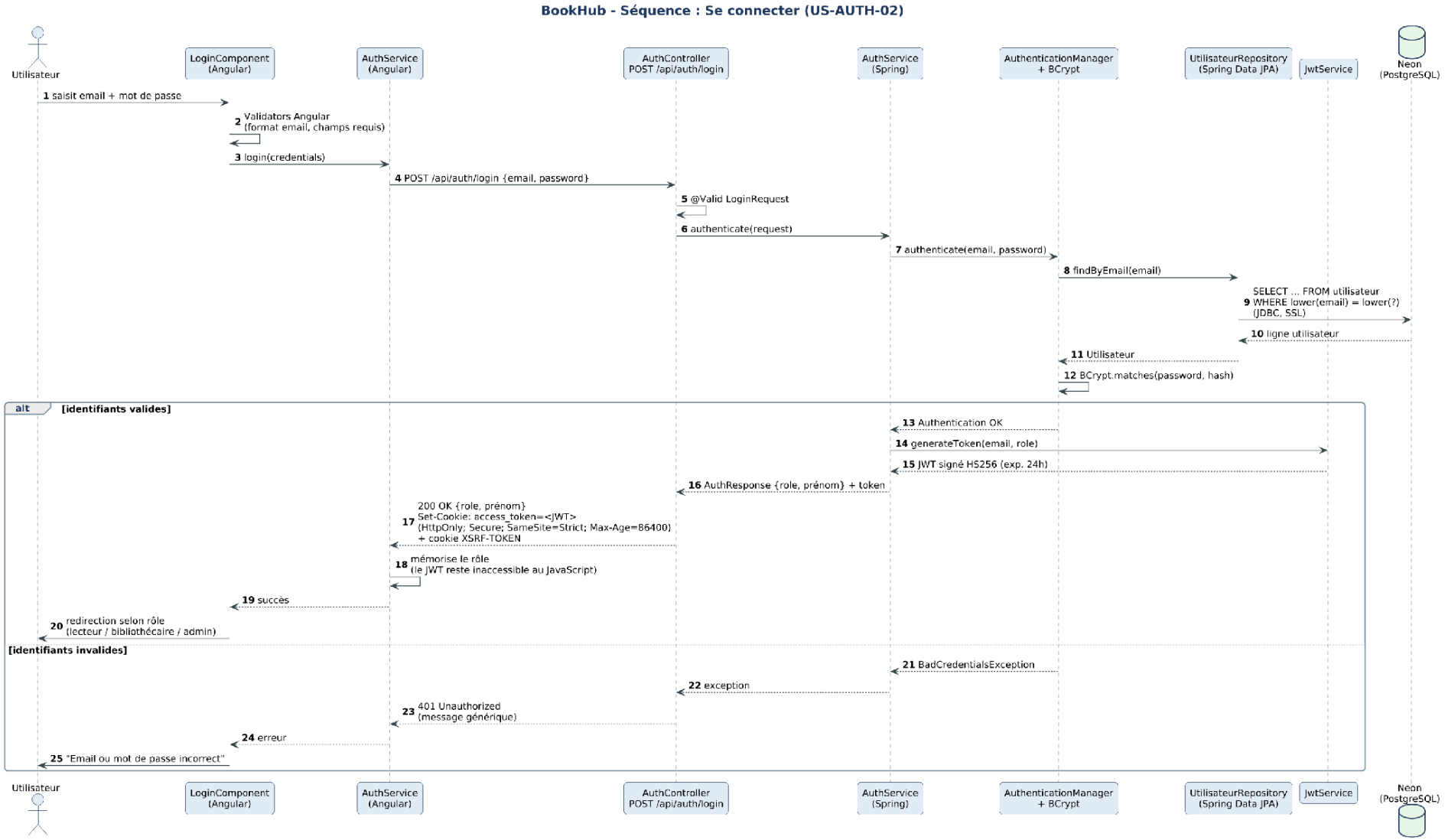
3.1 Diagramme de cas d'utilisation

Le diagramme reprend la matrice des droits : chaque rôle hérite des cas d'utilisation du rôle situé en dessous (Visiteur, puis USER, puis LIBRARIAN, puis ADMIN).



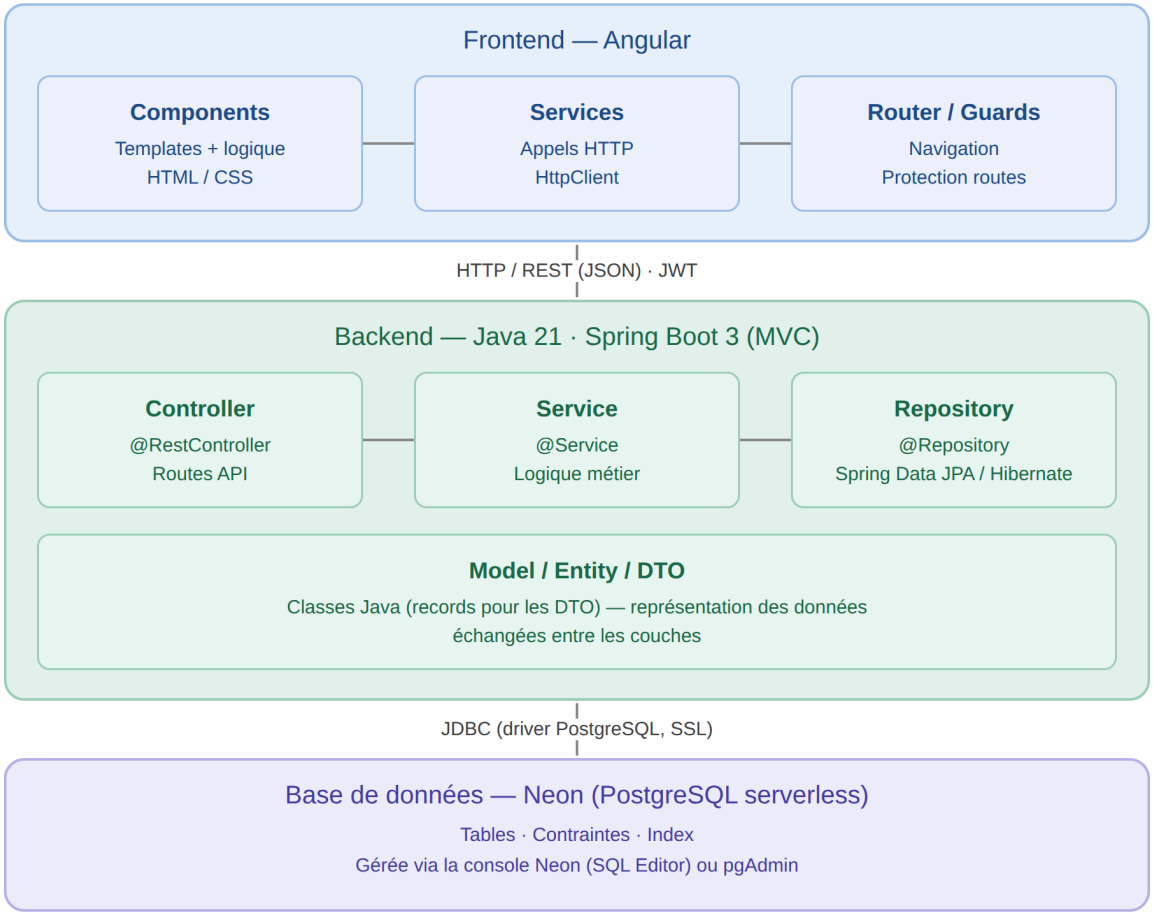
3.2 Diagramme de séquence : connexion

Ce diagramme détaille la connexion d'un utilisateur (US-AUTH-02), du formulaire Angular jusqu'à la base Neon, avec le cas d'identifiants invalides.



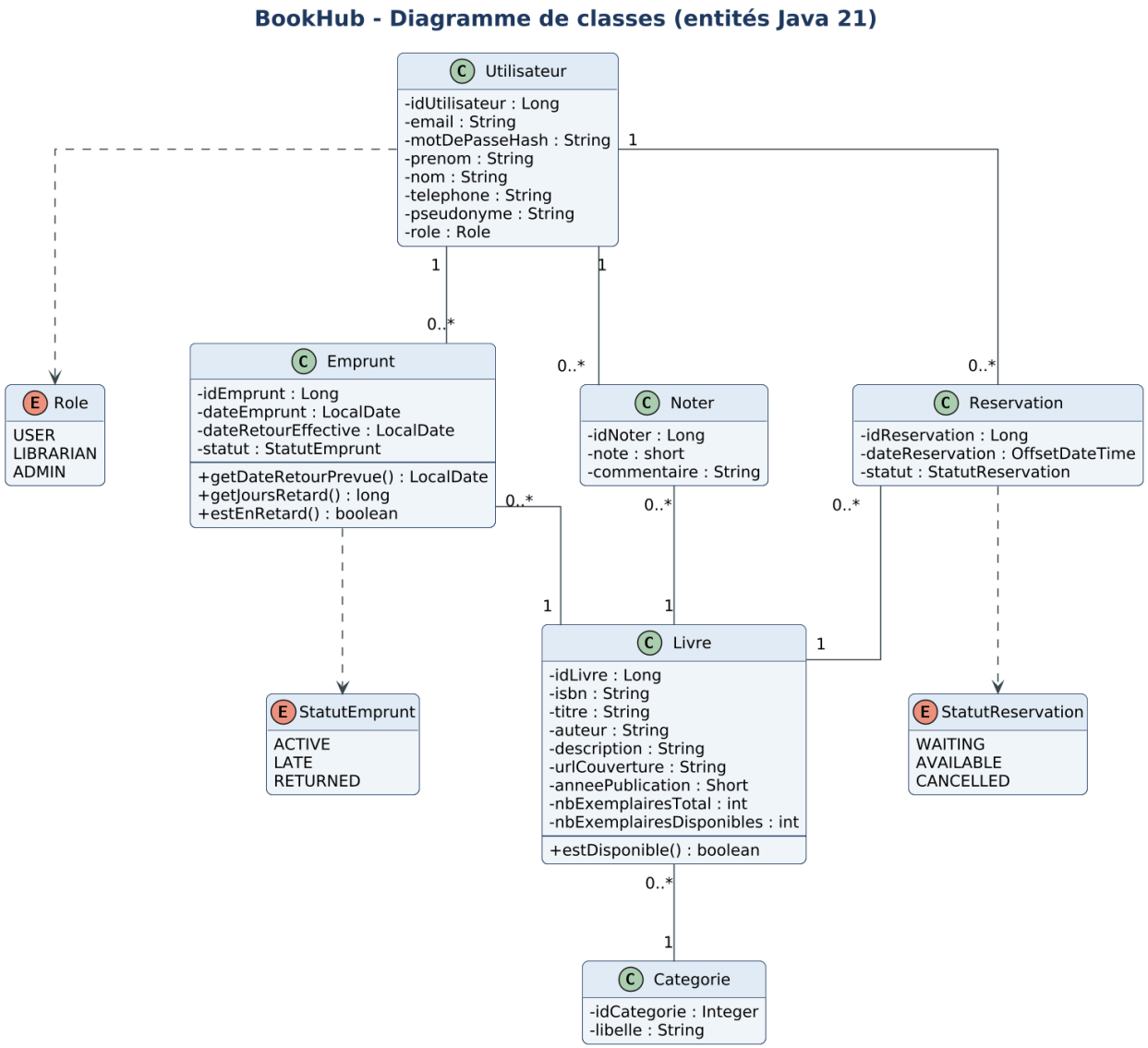
3.3 Diagramme d'architecture

L'application suit une architecture multicouche REST : le frontend Angular appelle l'API Spring Boot, qui accède à la base Neon par JDBC chiffré (SSL).



3.4 Diagramme de classes

Les entités Java reprennent les tables du modèle de données. Les valeurs calculées (date de retour prévue, retard, disponibilité) sont des méthodes et ne sont pas stockées en base.



4. Modèle de données

SGBD cible : Neon (PostgreSQL 16+). Les noms sont les mêmes dans le dictionnaire, le MCD, le MLD et le MPD.

4.1 Dictionnaire de données

Légende des types : AN = alphanumérique, N = numérique, D = date, DH = date et heure. PK = clé primaire, FK = clé étrangère, UQ = unique, NN = obligatoire.

Utilisateur

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_utilisateur	Identifiant de l'utilisateur	N	–	BIGINT IDENTITY	PK	3
email	Adresse e-mail, sert d'identifiant de connexion	AN	255	VARCHAR(255)	NN, UQ (insensible à la casse)	marie.durand@mail.fr
mot_de_passe_hash	Mot de passe chiffré (BCrypt, coût 12)	AN	100	VARCHAR(100)	NN	$2a$12$…
prenom	Prénom	AN	100	VARCHAR(100)	NN	Marie
nom	Nom de famille	AN	100	VARCHAR(100)	NN	Durand
telephone	Numéro de téléphone	AN	20	VARCHAR(20)	Facultatif	06 12 34 56 78
pseudonyme	Nom affiché sur les avis	AN	50	VARCHAR(50)	Facultatif	MarieLit
role	Rôle de l'utilisateur	AN	20	VARCHAR(20)	NN, défaut USER, valeurs USER / LIBRARIAN / ADMIN	USER

Categorie

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_categorie	Identifiant de la catégorie	N	–	INT IDENTITY	PK	1
libelle	Nom de la catégorie	AN	100	VARCHAR(100)	NN, UQ	Science-fiction

Livre

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_livre	Identifiant du livre	N	–	BIGINT IDENTITY	PK	1

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
isbn	Numéro ISBN-13	AN	17	VARCHAR(17)	NN, UQ	978-2-266-32026-0
titre	Titre du livre	AN	255	VARCHAR(255)	NN, index de recherche	Dune
auteur	Auteur du livre	AN	255	VARCHAR(255)	NN, index de recherche	Frank Herbert
description	Résumé du livre	AN	2000	VARCHAR(2000)	Facultatif	…
url_couverture	Lien vers l'image de couverture	AN	500	VARCHAR(500)	Facultatif	https://…
annee_publication	Année de publication	N	4	SMALLINT	Facultatif, entre 1450 et 2100	1965
nb_exemplaires_total	Nombre total d'exemplaires	N	–	INT	NN, défaut 1, supérieur ou égal à 0	3
nb_exemplaires_disponibles	Exemplaires disponibles à l'emprunt	N	–	INT	NN, entre 0 et nb_exemplaires_total, mis à jour à chaque emprunt et retour	2
id_categorie	Catégorie du livre	N	–	INT	NN, FK vers categorie	1

Emprunt

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_emprunt	Identifiant de l'emprunt	N	–	BIGINT IDENTITY	PK	1
date_emprunt	Date de début de l'emprunt	D	–	DATE	NN, défaut date du jour	2026-10-12
date_retour_effective	Date où le livre a été rendu	D	–	DATE	Vide tant que le livre n'est pas rendu, supérieure ou égale à date_emprunt	2026-10-20
statut	État de l'emprunt	AN	10	VARCHAR(10)	NN, défaut ACTIVE, valeurs ACTIVE / LATE / RETURNED ; RETURNED si et seulement si date_retour_effective est renseignée	ACTIVE
id_utilisateur	Lecteur qui emprunte	N	–	BIGINT	NN, FK vers utilisateur	3
id_livre	Livre emprunté	N	–	BIGINT	NN, FK vers livre	1

Reservation

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_reservation	Identifiant de la réservation	N	–	BIGINT IDENTITY	PK	1
date_reservation	Date et heure de la réservation, fixe l'ordre dans la file	DH	–	TIMESTAMPTZ	NN, défaut maintenant	2026-10-15 14:32

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
statut	État de la réservation	AN	10	VARCHAR(10)	NN, défaut WAITING, valeurs WAITING / AVAILABLE / CANCELLED (Attente / Dispo / Annulée)	WAITING
id_utilisateur	Lecteur qui réserve	N	–	BIGINT	NN, FK vers utilisateur	5
id_livre	Livre réservé	N	–	BIGINT	NN, FK vers livre	2

Une seule réservation active (WAITING ou AVAILABLE) par lecteur et par livre.

Noter

Code	Désignation	Type	Taille	Type SQL	Contraintes	Exemple
id_noter	Identifiant de l'avis (utilisé par PUT et DELETE /api/ratings/{id})	N	–	BIGINT IDENTITY	PK	1
note	Note attribuée	N	1	SMALLINT	NN, entre 1 et 5	4
commentaire	Avis écrit	AN	1000	VARCHAR(1000)	Facultatif	Un classique…
id_utilisateur	Lecteur qui donne l'avis	N	–	BIGINT	NN, FK vers utilisateur	3
id_livre	Livre noté	N	–	BIGINT	NN, FK vers livre	1

Un seul avis par lecteur et par livre (contrainte UNIQUE sur id_utilisateur + id_livre).

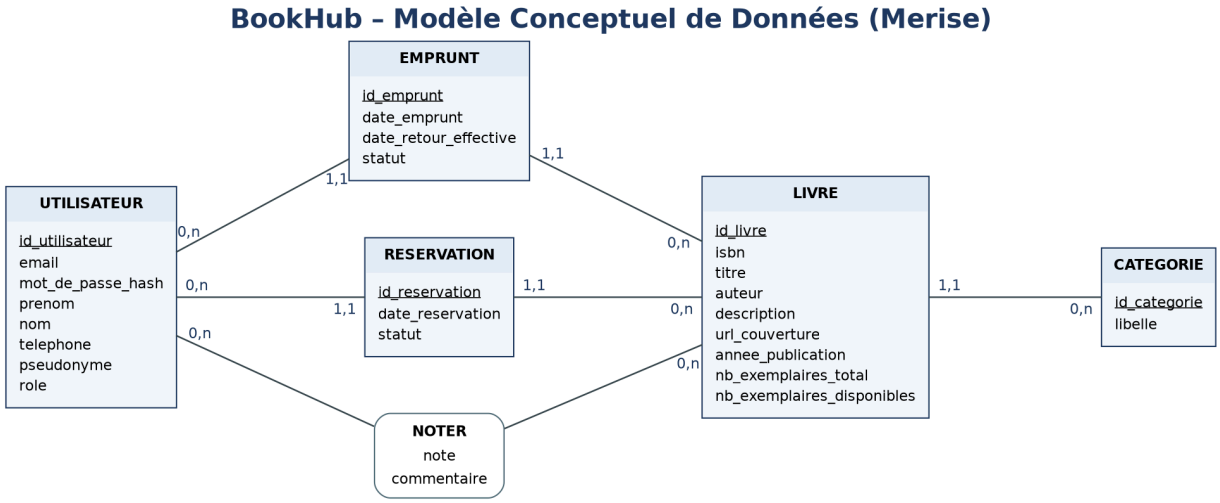
Données calculées

Ces données ne sont pas stockées en base. Elles sont calculées par requête.

Donnée	Calcul
Date de retour prévue	date_emprunt + 14 jours
Jours de retard	date du jour − (date_emprunt + 14), si l'emprunt n'est pas rendu
Rang dans la file d'attente	ordre des date_reservation au statut WAITING pour un même livre
Note moyenne d'un livre	moyenne des note de noter pour ce livre
Nombre d'avis d'un livre	nombre de lignes de noter pour ce livre

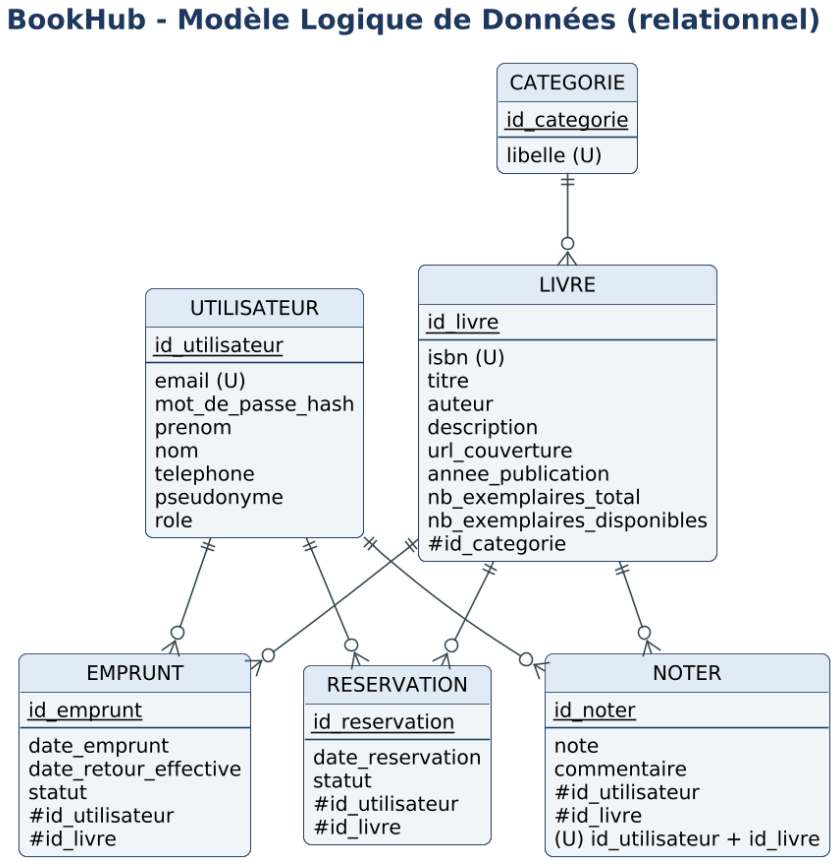
4.2 Modèle conceptuel de données (MCD)

Le MCD présente les entités et leurs relations avec les cardinalités. NOTER porte la note et le commentaire d'un lecteur sur un livre.



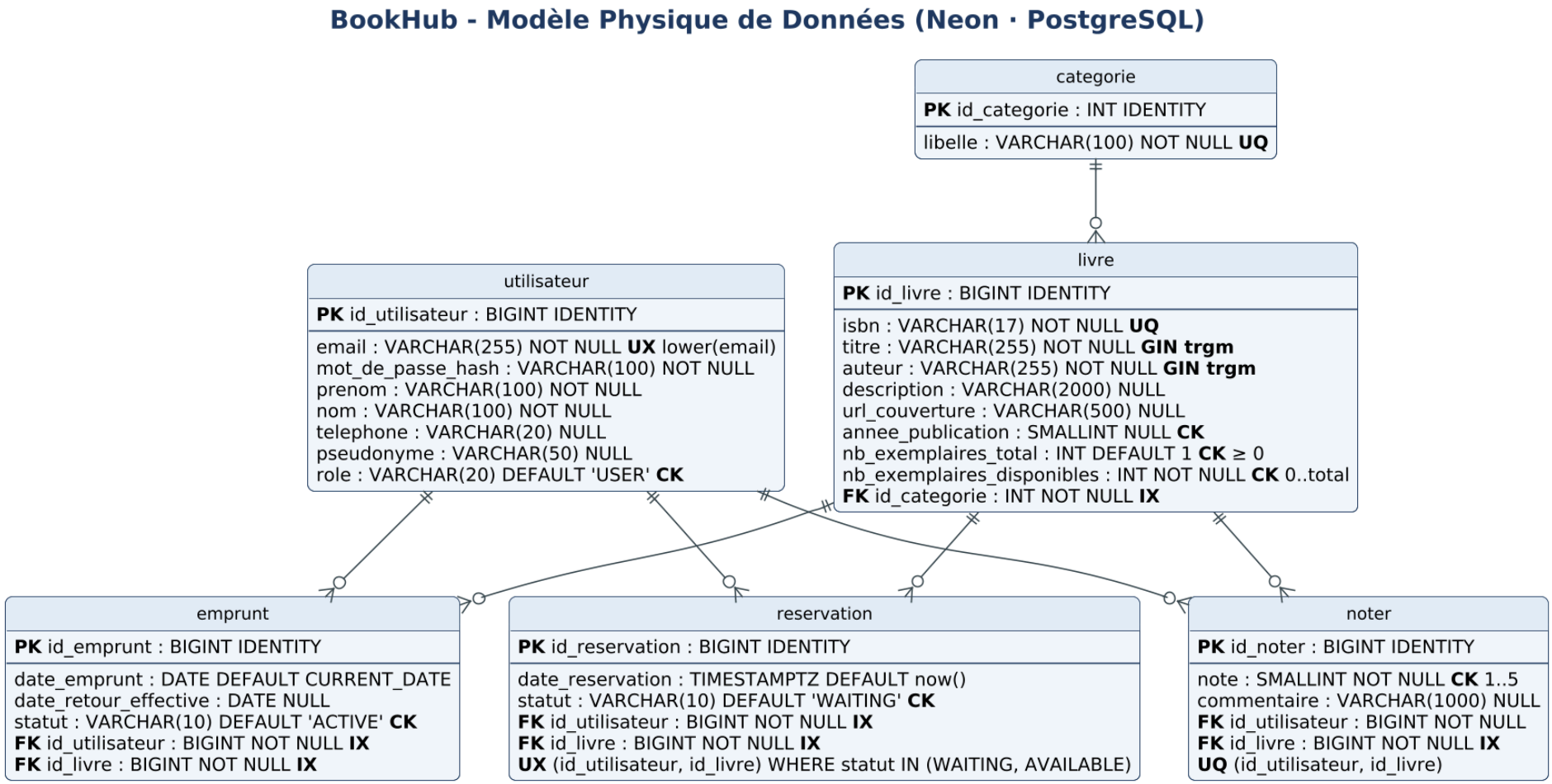
4.3 Modèle logique de données (MLD)

Le MLD traduit le MCD en tables relationnelles. La table NOTER reçoit un identifiant id_noter, nécessaire aux routes PUT et DELETE /api/ratings/{id} du cahier des charges.



4.4 Modèle physique de données (MPD)

Le MPD précise les types PostgreSQL, les contraintes (PK, FK, UQ, CK) et les index utilisés par Neon.



4.5 Triggers et fonctions PostgreSQL

Le cahier des charges (§ 3.2) demande des procédures stockées et des triggers « présents et pertinents ». Dans BookHub, ils servent à deux choses : garantir le stock d'exemplaires directement dans la base, quel que soit le code qui l'appelle, et calculer les indicateurs des tableaux de bord sans faire remonter toutes les lignes dans l'application (objectif de performance < 2 s et d'éco-conception).

Sous PostgreSQL, il n'existe pas de procédure stockée au sens SQL Server : on écrit des fonctions en SQL ou en PL/pgSQL. Un trigger appelle toujours une fonction qui renvoie le type trigger.

Triggers

Trigger (fonction)	Déclenchement	Rôle	Règle
T1 emprunt_ai_stock (trg_emprunt_stock)	AFTER INSERT sur emprunt	Retire 1 aux exemplaires disponibles du livre emprunté. S'il n'en reste aucun, la contrainte ck_livre_exemplaires (disponibles ≥ 0) fait échouer l'emprunt.	RG-LOAN-05
T2 emprunt_au_retour (trg_emprunt_retour)	AFTER UPDATE OF statut sur emprunt, quand le statut passe à RETURNED	Rajoute 1 aux exemplaires disponibles du livre rendu.	RG-LOAN-05 RG-LOAN-06
T3 livre_bu_exemplaires (trg_livre_exemplaires)	BEFORE UPDATE OF nb_exemplaires_total sur livre	Reporte l'écart sur les exemplaires disponibles quand le bibliothécaire modifie le nombre total. Refuse la modification si le total devient inférieur au nombre de livres empruntés.	US-BOOK-04

Fonctions

Fonction	Retour	Rôle	Utilisée par
F1 fn_marquer_retards()	INT (nombre d'emprunts passés en retard)	Passe au statut LATE les emprunts ACTIVE dont la date de retour prévue (date_emprunt + 14 jours) est dépassée.	Tâche @Scheduled quotidienne RG-LOAN-02, RG-LOAN-03
F2 fn_stats_bibliotheque()	1 ligne : nb livres, nb exemplaires, emprunts actifs, retards, réservations en attente	Calcule les indicateurs chiffrés en une seule requête.	Dashboards bibliothécaire et admin US-DASH-02, US-DASH-03
F3 fn_top_livres(p_top = 10, p_mois = 12)	Liste : id, titre, auteur, nb emprunts	Classe les livres les plus empruntés sur la période.	Widget « Top 10 livres » US-DASH-02
F4 fn_emprunts_par_mois(p_mois = 12)	Liste : mois, nb emprunts	Compte les emprunts mois par mois.	Graphique d'évolution US-DASH-02

F5 fn_anonymiser_utilisateur(p_id_utilisateur)	Aucun (void)	Droit à l'oubli : refuse s'il reste des emprunts en cours, supprime les réservations et les avis, puis anonymise le compte. Les emprunts sont conservés pour les statistiques.	Suppression de compte RG-AUTH-06, US-AUTH-03

Appel depuis Spring Boot

Les triggers n'ont pas besoin d'être appelés : ils s'exécutent seuls lors des INSERT et UPDATE faits par JPA. Après un emprunt ou un retour, le service relit le livre pour obtenir le stock à jour. Les fonctions sont appelées par des requêtes natives dans les repositories :

```
@Query(value = "SELECT * FROM fn_top_livres(:top, :mois)", nativeQuery = true)
List<TopLivreDto> findTopLivres(@Param("top") int top, @Param("mois") int mois);
@Query(value = "SELECT fn_marquer_retards()", nativeQuery = true)
int marquerRetards();
```

Les règles qui dépendent du contexte de l'utilisateur (3 emprunts maximum, 5 réservations, blocage en cas de retard, droit de noter) restent dans les services Spring, où elles sont testées en JUnit et renvoient des messages d'erreur clairs